

The Alberta Buck - Formal Correctness Proofs (DRAFT v0.9)

Rerandomization, Re-Encryption, and Notes Spend Predicates

Perry Kundert

2026-04-22



The Alberta Buck identity system (Identity, Examples) rests on two cryptographic transformations: the *Identity Fountain* rerandomizes a Pointcheval-Sanders signature to produce an unlinkable credential, and the **approve** handshake re-encrypts the credential's ElGamal ciphertext under the counterparty's public key with a Chaum-Pedersen proof of equality. Everything downstream – bilateral transfers, the BUCK Notes spend circuits (Notes), Identity Guardian authorizations, regulatory recovery – inherits its correctness from these two primitives.

This document states and proves the correctness properties that the rest of the series relies on:

- **Theorem 1:** rerandomization preserves PS signature validity (completeness).
- **Theorem 2:** a uniform random rerandomization scalar yields a signature statistically identical to a fresh one, independent of the original (unlinkability).
- **Theorem 3:** an honest Chaum-Pedersen prover always verifies (completeness).
- **Theorem 4:** from two accepting transcripts with distinct challenges an efficient extractor recovers the witness (special soundness, 2-extractability).
- **Theorem 5:** the sigma-protocol transcript is perfectly simulatable given only public inputs and the challenge (HVZK).
- **Theorem 6:** combined, an accepted on-chain Chaum-Pedersen proof implies that the re-encrypted ciphertext carries the same issuer-certified identity point $M = m \cdot G$ as the registered credential (end-to-end re-encryption correctness).
- **Theorem 7:** ElGamal re-randomization at note mint preserves the recipient's encrypted identity point under their registered key (BUCK Notes mint correctness).

- **Theorem 8:** an accepted A-spend SNARK on a noteTree commitment implies, via Chaum-Pedersen equality, that the depositor’s registered credential and the note’s ciphertext encrypt the same identity point under the *same* public key – the pk_{rec} that was live at mint time. A-notes are bound to that key pair: if sk_{rec} is lost, Identity re-issuance to a fresh keypair (sk', pk') on the same M does not restore spend capability.
- **Theorem 9:** an accepted B-spend SNARK certifies (i) a registered depositor knowing the bearer secret ρ and (ii) a Schnorr-signed public-issuer attestation on the commitment.
- **Theorem 10:** the A-spend nullifier (keyed on M_{rec}) is a deterministic function of (M_{rec}, ρ) alone and is not precomputable by the issuer; the B-spend nullifier (keyed on ρ) is publicly derivable – the bearer-cheque property. Domain separation prevents cross-flavor collision.
- **Theorem 11:** Notes preserve the BUCK mutual-decryptability invariant – both counterparties recover the other’s identity point in every accepted spend.

Every theorem is accompanied by an executable `py_ecc` sanity check on BN254. The proofs assume only the q-SDH assumption (for PS signature unforgeability), the discrete logarithm assumption on $\text{alt_bn128 } G_1$, the random oracle model (for the Fiat-Shamir transform and Poseidon hashing), and knowledge soundness of the underlying SNARK proof system (for Theorems 8-9) – all standard assumptions that Ethereum’s security or any zk-SNARK rollup already requires. (PDF, Text)

Contents

1	Preliminaries	4
1.1	Notation	4
1.2	Hardness Assumptions	4
2	Part I: Pointcheval-Sanders Signature Rerandomization	5
2.1	The Construction	5
2.2	Theorem 1 (Completeness of Rerandomization)	5
2.3	Theorem 2 (Statistical Unlinkability)	5
2.4	Sanity Check (py_ecc on BN254)	6
3	Part II: Chaum-Pedersen Re-Encryption Correctness	8
3.1	Setup	8
3.2	The Statement	8
3.3	Protocol (Fiat-Shamir)	8
3.4	Theorem 3 (Completeness)	9
3.5	Theorem 4 (Special Soundness / 2-Extractability)	9
3.6	Theorem 5 (Honest-Verifier Zero-Knowledge)	10
3.7	Sanity Check (py_ecc on BN254)	10
4	Part III: End-to-End Re-Encryption Correctness	13
4.1	Chain of Trust	13
4.2	Theorem 6 (End-to-End Re-Encryption Correctness)	13
4.3	Why This Suffices for Every Downstream Use Case	13
5	Part IV: BUCK Notes Spend Predicates (A1, A2, B1)	15
5.1	Setup	15
5.2	Theorem 7 (Mint Re-randomization Correctness)	16
5.3	Theorem 8 (A-Spend Soundness)	17
5.4	Theorem 9 (B-Spend Soundness)	18
5.5	Theorem 10 (Nullifier Construction and Double-Spend Prevention)	19
5.6	Theorem 11 (Mutual-Decryptability Preservation)	20
5.7	Sanity Check (py_ecc on BN254)	21
5.8	Worked Example: End-to-End A1 Note Transfer (SNARK Relations)	23
5.9	Mint-Circuit Status: Phase 7-bis Closure of the Earlier Open Questions	25
5.9.1	Closure 1 (Field-Overflow / Value-Conservation Range Check) – <i>Discharged in circuit</i>	25
5.9.2	Closure 2 (Single <code>transferFrom</code> $\leftrightarrow$ N Commitments) – <i>Discharged by SNARK binding</i>	25
5.9.3	Closure 3 (Spend-Circuit Trio) – <i>Mint half discharged; nullifier + spend-side value conservation deferred to <code>spend.circom</code></i>	25
6	Security Assumptions Summary	27
7	Implementation Notes	28
7.1	What the Contract Must Enforce	28
7.2	What the Prover Must Enforce	28
7.3	Scope Not Covered Here	28

1 Preliminaries

1.1 Notation

Symbol	Meaning
q	Prime order of G_1, G_2, G_T on BN254
G, g_2	Generators of G_1 and G_2
$e : G_1 \times G_2 \rightarrow G_T$	Bilinear, non-degenerate pairing (type-3)
O	Point at infinity (group identity)
$\mathbb{Z}_q^*$	Non-zero scalars mod q
$(x, y)/(X, Y)$	Issuer secret / public key: $X = xg_2, Y = yg_2$
$m \in \mathbb{Z}_q$	Identity scalar, $m = H(\text{identity_data}) \bmod q$
$M = m \cdot G \in G_1$	Identity point (the value every ElGamal ciphertext encrypts)
$\sigma = (\sigma_1, \sigma_2) \in G_1^2$	PS signature: $\sigma_1 = h, \sigma_2 = (x + my)h$
(sk, pk)	Identity key pair on G_1 : $pk = sk \cdot G$
$E = (R, C) \in G_1^2$	ElGamal ciphertext: $R = rG, C = M + r \cdot pk$
$H(\cdot)$	Cryptographic hash modelled as a random oracle (Fiat-Shamir)

All group operations in this document are written additively. BN254 has cofactor 1 on G_1 , so every non-identity $P \in G_1$ has order q ; there are no small-subgroup elements.

1.2 Hardness Assumptions

Four standard assumptions underpin the theorems in this document. None is specific to Alberta Buck – each one is already required by Ethereum’s own security model or by any pairing-based / zk-SNARK construction built on the `ecPairing` precompile.

1. **Discrete Logarithm (DLog) on G_1** : given $P, Q \in G_1$, finding $\alpha \in \mathbb{Z}_q$ with $Q = \alpha P$ is infeasible. Protects ElGamal IND-CPA security, Schnorr unforgeability, and Chaum-Pedersen soundness.
2. **q-Strong Diffie-Hellman (q-SDH)**: the PS signature scheme is existentially unforgeable under chosen-message attack (EUF-CMA) assuming q-SDH¹. We use this only as a **stated** assumption about the PS primitive; none of our proofs below reduce to it directly.
3. **Random Oracle Model (ROM)**: the Fiat-Shamir transform is applied to sigma protocols whose security is proved in the ROM, with the hash function H modelled as a random oracle bound to all public inputs and context (caller address, chain id, etc.). Poseidon (commitment hash and nullifier PRF) is also modelled as a random oracle for the BUCK Notes proofs in Part IV.
4. **Knowledge soundness of the underlying zk-SNARK proof system** (Part IV only): the SNARK system used for BUCK Notes spend circuits (Groth16, PLONK, or any knowledge-sound construction over BN254) admits a polynomial-time witness extractor on every accepting proof. This is a system-specific assumption depending on the chosen scheme’s trusted setup or universal setup.

The rerandomization proofs (Part I) are *unconditional*: Theorem 1 follows from bilinearity alone, and Theorem 2 is information-theoretic. The Chaum-Pedersen proofs (Part II) are interactive-sigma-protocol facts; the NIZK corollary (Part II, end) invokes the ROM. Part IV (Notes spend predicates) additionally invokes assumption (4) – knowledge soundness of the underlying SNARK system – to lift the sigma-protocol guarantees of Theorems 4-5 into the SNARK setting.

¹Pointcheval, D. and Sanders, O. "Short Randomizable Signatures." CT-RSA 2016, LNCS 9610, pp. 111-126. Theorem 4.2 establishes EUF-CMA security under the q-SDH assumption in bilinear groups.

2 Part I: Pointcheval-Sanders Signature Rerandomization

2.1 The Construction

A Pointcheval-Sanders signature on $m \in \mathbb{Z}_q$ is a pair $\sigma = (\sigma_1, \sigma_2) \in G_1^2$ with $\sigma_1 \neq O$ satisfying the pairing equation

$$e(\sigma_1, X + m \cdot Y) = e(\sigma_2, g_2).$$

The issuer produces σ by choosing a fresh $h \in G_1 \setminus \{O\}$ and setting $\sigma_1 = h$, $\sigma_2 = (x + my)h$. *Rerandomization* is the operation

$$\sigma' = (t \cdot \sigma_1, t \cdot \sigma_2) \quad t \in \mathbb{Z}_q^*.$$

2.2 Theorem 1 (Completeness of Rerandomization)

Statement. For every valid PS signature σ on m under issuer key (X, Y) , and for every $t \in \mathbb{Z}_q^*$, the pair $\sigma' = (t\sigma_1, t\sigma_2)$ is also a valid PS signature on m under the same issuer key.

Proof. By bilinearity of e :

$$\begin{aligned} e(\sigma'_1, X + mY) &= e(t\sigma_1, X + mY) && \text{(definition)} \\ &= e(\sigma_1, X + mY)^t && \text{(bilinearity)} \\ &= e(\sigma_2, g_2)^t && \text{(original validity)} \\ &= e(t\sigma_2, g_2) && \text{(bilinearity)} \\ &= e(\sigma'_2, g_2). \end{aligned}$$

Non-degeneracy: G_1 has prime order q , so $t\sigma_1 = O$ iff $t \equiv 0 \pmod{q}$ or $\sigma_1 = O$. Both are excluded by hypothesis ($t \in \mathbb{Z}_q^*$ and $\sigma_1 \neq O$ by PS validity). Therefore $\sigma'_1 \neq O$ and the pair satisfies the full PS validity predicate. ■

Implementation guard. The registration contract MUST enforce $\sigma'_1 \neq O$. Without this check, a prover can submit $\sigma' = (O, O)$ (obtainable by picking $t = 0$); the pairing equation then reduces to $1 = 1$ in G_T for every m , allowing a fabricated credential to pass registration. Theorem 1's completeness argument explicitly depends on $t \neq 0$; the contract's guard is the runtime witness of this hypothesis.

2.3 Theorem 2 (Statistical Unlinkability)

Statement. Let σ be any fixed valid PS signature on m . If t is sampled uniformly from $\mathbb{Z}_q^*$, then the distribution of $\sigma' = (t\sigma_1, t\sigma_2)$ is identical to the distribution of $\text{PS.Sign}(sk_{\text{issuer}}, m)$ for h sampled uniformly from $G_1 \setminus \{O\}$, and is *independent* of σ .

Proof. Since G_1 is cyclic of prime order q and $\sigma_1 \neq O$, the map

$$\varphi_\sigma : \mathbb{Z}_q^* \rightarrow G_1 \setminus \{O\}, \quad t \mapsto t\sigma_1$$

is a bijection (it is the restriction to non-zero scalars of the group isomorphism $\mathbb{Z}_q \rightarrow \langle \sigma_1 \rangle = G_1$). Hence σ'_1 is uniform on $G_1 \setminus \{O\}$ regardless of the specific σ_1 we started with.

Given $\sigma'_1 = h'$, the second component is forced: $\sigma'_2 = t\sigma_2 = t(x + my)\sigma_1 = (x + my)(t\sigma_1) = (x + my)h'$. That is exactly what a fresh PS.Sign with random h' would output.

So the joint distribution of σ' matches the fresh-signature distribution, and the map $\sigma \mapsto \sigma'$ destroys all information about σ beyond the fact that both sign the same m . This is *statistical* (information-theoretic) unlinkability, not merely computational. ■

Remark. The argument uses only that G_1 has prime order; no cryptographic assumption is invoked. A cofactor > 1 would leak subgroup membership of σ_1 ; the prime-order property of BN254 G_1 is therefore load-bearing.

2.4 Sanity Check (py_ecc on BN254)

```
import secrets, hashlib
from py_ecc.bn128 import bn128_curve as bc
from py_ecc.bn128 import pairing

ORDER = bc.curve_order
G1, G2 = bc.G1, bc.G2
multiply, add, neg, eq = bc.multiply, bc.add, bc.neg, bc.eq

def rand():
    return secrets.randbelow(ORDER - 1) + 1

# Issuer key pair
x, y = rand(), rand()
X, Y = multiply(G2, x), multiply(G2, y)

# Alice's identity and PS signature sigma = (h, (x + m y) h)
m = int(hashlib.sha256(b"Alice Johnson | Alberta | AIC-2026").hexdigest(), 16) % ORDER
h = multiply(G1, rand())
sigma = (h, multiply(h, (x + m * y) % ORDER))

# Rerandomize with t != 0
t = rand()
sigma_p = (multiply(sigma[0], t), multiply(sigma[1], t))

# --- Theorem 1: completeness ---
lhs = pairing(add(X, multiply(Y, m)), sigma_p[0])
rhs = pairing(G2, sigma_p[1])
assert lhs == rhs, "Theorem 1 FAILED: rerandomized signature does not verify"
assert not eq(sigma_p[0], bc.Z1), "Theorem 1 guard: sigma'_1 == 0"
print("Theorem 1 (completeness):          PASS")

# --- Theorem 2: fresh rerandomization looks like a fresh signing ---
# Two independent rerandomizations should be indistinguishable from two
# independent fresh signings. We check both give valid signatures on m
# whose sigma_1 components are independent uniform-looking elements.
t2 = rand()
sigma_p2 = (multiply(sigma[0], t2), multiply(sigma[1], t2))
assert not eq(sigma_p[0], sigma_p2[0]), "distinct rerandomizations collide"
lhs2 = pairing(add(X, multiply(Y, m)), sigma_p2[0])
rhs2 = pairing(G2, sigma_p2[1])
assert lhs2 == rhs2
print("Theorem 2 (unlinkability):          PASS (two rerandomizations both verify, distinct)")

# --- Theorem 1 guard: the trivial signature (t = 0) is degenerate ---
zero_sig = (bc.Z1, bc.Z1)
lhs0 = pairing(add(X, multiply(Y, m)), zero_sig[0]) # pairing with 0 -> 1_GT
rhs0 = pairing(G2, zero_sig[1]) # pairing with 0 -> 1_GT
# The pairing equation holds for ANY m -- this is why the guard matters
assert lhs0 == rhs0
# But the registration contract rejects it because sigma'_1 == 0:
rejected = eq(zero_sig[0], bc.Z1)
assert rejected
print("Theorem 1 guard (sigma'_1 != 0):    PASS (trivial signature rejected by guard)")
```

Theorem 1 (completeness): PASS

Theorem 2 (unlinkability):	PASS (two rerandomizations both verify, distinct)
Theorem 1 guard ($\sigma'_1 \neq 0$):	PASS (trivial signature rejected by guard)

How to read the output: Three PASS lines. The first confirms Theorem 1's pairing identity on a random rerandomization. The second confirms two fresh rerandomizations are distinct and both verify (a minimal demonstration of the unlinkable-yet-valid property). The third demonstrates *why* the $\sigma'_1 \neq 0$ check is mandatory: the pairing equation is trivially satisfied by (O, O) for any m , so only the explicit guard prevents this degenerate signature from passing.

DRAFT

3 Part II: Chaum-Pedersen Re-Encryption Correctness

3.1 Setup

Alice holds a credential whose ElGamal ciphertext is registered on chain and was verified at registration time against her PS signature:

$$E_{\text{alice}} = (R_a, C_a) = (r_a \cdot G, M + r_a \cdot pk_{\text{alice}}), \quad pk_{\text{alice}} = sk_{\text{alice}} \cdot G.$$

To identify herself to Bob she re-encrypts under pk_{bob} with fresh randomness r_b :

$$E_{\text{bob}} = (R_b, C_b) = (r_b \cdot G, M + r_b \cdot pk_{\text{bob}}).$$

She then produces a non-interactive Chaum-Pedersen proof that E_{alice} and E_{bob} encrypt the *same* message point M – without revealing M , m , sk_{alice} , or r_b . The BUCK contract reads E_{alice} from storage (not calldata) and verifies the proof.

3.2 The Statement

Alice proves knowledge of (sk_{alice}, r_b) satisfying the three simultaneous relations

$$pk_{\text{alice}} = sk_{\text{alice}} \cdot G \tag{S1}$$

$$R_b = r_b \cdot G \tag{S2}$$

$$C_a - sk_{\text{alice}} \cdot R_a = C_b - r_b \cdot pk_{\text{bob}}. \tag{S3}$$

(S1) binds her to the registered public key; (S2) binds r_b to the ciphertext she submits; (S3) states that the ElGamal-decrypt of E_{alice} equals the ElGamal-decrypt of E_{bob} under their respective secret keys.

3.3 Protocol (Fiat-Shamir)

Commit. Prover samples $k_1, k_2 \leftarrow \mathbb{Z}_q$ and computes

$$T_1 = k_1 \cdot G, \quad T_2 = k_2 \cdot G, \quad T_3 = k_2 \cdot pk_{\text{bob}} - k_1 \cdot R_a.$$

Challenge.

$$e = H(G, pk_{\text{alice}}, pk_{\text{bob}}, R_a, C_a, R_b, C_b, T_1, T_2, T_3, \text{msg.sender}, \text{spender}, \text{chainid}).$$

$$\textbf{Respond. } s_1 = k_1 - e \cdot sk_{\text{alice}} \pmod{q}, \quad s_2 = k_2 - e \cdot r_b \pmod{q}.$$

Verify. The contract recomputes e from the public inputs and the prover-supplied commitments, then checks

$$s_1 \cdot G + e \cdot pk_{\text{alice}} \stackrel{?}{=} T_1 \tag{V1}$$

$$s_2 \cdot G + e \cdot R_b \stackrel{?}{=} T_2 \tag{V2}$$

$$s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a) \stackrel{?}{=} T_3. \tag{V3}$$

3.4 Theorem 3 (Completeness)

Statement. An honestly generated transcript satisfies (V1), (V2), (V3).

Proof. (V1) and (V2) are standard Schnorr identities:

$$\begin{aligned} s_1 G + e \cdot pk_{\text{alice}} &= (k_1 - e \cdot sk_{\text{alice}})G + e \cdot sk_{\text{alice}}G = k_1 G = T_1, \\ s_2 G + e \cdot R_b &= (k_2 - er_b)G + er_b G = k_2 G = T_2. \end{aligned}$$

For (V3), substitute s_1, s_2 and rearrange:

$$\begin{aligned} s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e(C_b - C_a) &= (k_2 - er_b)pk_{\text{bob}} - (k_1 - e \cdot sk_{\text{alice}})R_a + e(C_b - C_a) \\ &= [k_2 pk_{\text{bob}} - k_1 R_a] + e[sk_{\text{alice}}R_a - r_b \cdot pk_{\text{bob}} + C_b - C_a] \\ &= T_3 + e[-(C_a - sk_{\text{alice}}R_a) + (C_b - r_b \cdot pk_{\text{bob}})] \\ &= T_3 + e \cdot [-M + M] \\ &= T_3. \end{aligned}$$

where the penultimate line uses the witness relations from the honest setup, and the final line uses (S3). ■

3.5 Theorem 4 (Special Soundness / 2-Extractability)

Statement. Given two accepting transcripts $(T_1, T_2, T_3, e, s_1, s_2)$ and $(T_1, T_2, T_3, e', s'_1, s'_2)$ with $e \neq e'$ (and identical public inputs), there is an efficient algorithm that outputs a witness (sk^*, r^*) satisfying (S1), (S2), (S3).

Proof (extractor). Subtract (V1) for the two transcripts:

$$(s_1 - s'_1)G + (e - e') \cdot pk_{\text{alice}} = O \implies pk_{\text{alice}} = \frac{s'_1 - s_1}{e - e'} \cdot G.$$

Set $sk^* := (s'_1 - s_1)(e - e')^{-1} \bmod q$; this is well-defined because q is prime and $e \neq e'$. By construction (S1) holds. Similarly from (V2), $r^* := (s'_2 - s_2)(e - e')^{-1} \bmod q$ satisfies (S2): $R_b = r^*G$.

For (S3), subtract (V3) for the two transcripts:

$$(s_2 - s'_2) \cdot pk_{\text{bob}} - (s_1 - s'_1) \cdot R_a + (e - e')(C_b - C_a) = O.$$

Substitute $s_2 - s'_2 = -(e - e')r^*$ and $s_1 - s'_1 = -(e - e')sk^*$:

$$-(e - e')r^* \cdot pk_{\text{bob}} + (e - e')sk^* \cdot R_a + (e - e')(C_b - C_a) = O.$$

Dividing through by the nonzero scalar $(e - e')$:

$$sk^* \cdot R_a - r^* \cdot pk_{\text{bob}} + (C_b - C_a) = O \iff C_a - sk^* \cdot R_a = C_b - r^* \cdot pk_{\text{bob}},$$

which is (S3). Therefore (sk^*, r^*) is a valid witness, and the extractor runs in the time of four field inversions and a constant number of group operations. ■

Remark (implication for cheating provers). Special soundness lifts, via the forking lemma², to a reduction from any efficient prover that convinces the verifier on a false statement to an efficient solver for discrete logarithms on G_1 . Under the DLog assumption on alt_bn128 , no such prover exists. In particular, Alice cannot produce a valid proof for E_{bob} encrypting any $M' \neq M$ – not merely "hard to compute," but infeasible under a standard assumption Ethereum already requires.

3.6 Theorem 5 (Honest-Verifier Zero-Knowledge)

Statement. There is an efficient simulator $\mathcal{S}$ that, given only the public inputs and a challenge e drawn from the verifier's distribution, outputs a transcript $(T_1, T_2, T_3, e, s_1, s_2)$ distributed *identically* to an honest prover's transcript conditioned on that challenge.

Proof (simulator). $\mathcal{S}$ samples $s_1, s_2 \leftarrow \mathbb{Z}_q$ uniformly, then *defines*

$$\begin{aligned} T_1 &:= s_1 \cdot G + e \cdot pk_{\text{alice}}, \\ T_2 &:= s_2 \cdot G + e \cdot R_b, \\ T_3 &:= s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a). \end{aligned}$$

Equations (V1), (V2), (V3) hold by construction, so the transcript is accepting.

Distribution match. In the real protocol, k_1, k_2 are uniform on $\mathbb{Z}_q$; the honest responses $s_i = k_i - e \cdot w_i$ are therefore uniform on $\mathbb{Z}_q$ (shift by a constant). The commitments T_1, T_2, T_3 are *deterministic functions* of the s_i and the public inputs: (V1), (V2), (V3) each uniquely determine one T_i from the corresponding equation. The simulator samples s_i uniformly and computes the T_i via the same equations – the joint distribution over $(T_1, T_2, T_3, s_1, s_2)$ is identical. ■

Corollary (NIZK via Fiat-Shamir in the ROM). Model H as a programmable random oracle. The Fiat-Shamir transform replaces the verifier's challenge with $e = H(\text{public inputs}, T_1, T_2, T_3, \text{context})$. Standard arguments³ promote:

- HVZK $\Rightarrow$ NIZK (the simulator programs H so that $H(\text{public inputs}, T_1, T_2, T_3, \dots) = e$ for the (T_i) it produced);
- special soundness $\Rightarrow$ proof-of-knowledge soundness (the forking lemma rewinds the oracle).

The binding of `msg.sender`, `spender`, `chainid` into the hash input is standard domain separation that prevents cross-context replay.

3.7 Sanity Check (py_ecc on BN254)

```
# Build registered credential E_alice for Alice
sk_alice = rand()
pk_alice = multiply(G1, sk_alice)
r_a      = rand()
R_a      = multiply(G1, r_a)
```

²Pointcheval, D. and Stern, J. "Security Arguments for Digital Signatures and Blind Signatures." Journal of Cryptology 13(3),

1. The forking lemma: an adversary that convinces a sigma-protocol verifier with non-negligible probability can be rewound to produce two accepting transcripts with distinct challenges, from which the witness is extracted.

³Fiat, A. and Shamir, A. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems." CRYPTO 1986. The transform replacing an interactive verifier's challenge with a hash of the transcript, analyzed in the random oracle model of Bellare and Rogaway (CCS 1993).

```

M          = multiply(G1, m)                                # identity point M = m * G
C_a        = add(M, multiply(pk_alice, r_a))

# Bob's identity public key (Alice knows this from the registry)
sk_bob = rand()
pk_bob = multiply(G1, sk_bob)

# Re-encryption for Bob: E_bob = (r_b * G, M + r_b * pk_bob)
r_b = rand()
R_b = multiply(G1, r_b)
C_b = add(M, multiply(pk_bob, r_b))

def cp_prove(k1, k2):
    T1 = multiply(G1, k1)
    T2 = multiply(G1, k2)
    T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
    tag = "|".join(str(v) for v in
                    [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b, T1, T2, T3])
    e = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
    s1 = (k1 - e * sk_alice) % ORDER
    s2 = (k2 - e * r_b) % ORDER
    return (T1, T2, T3, e, s1, s2)

def cp_verify(T1, T2, T3, e, s1, s2):
    v1 = eq(add(multiply(G1, s1), multiply(pk_alice, e)), T1)
    v2 = eq(add(multiply(G1, s2), multiply(R_b, e)), T2)
    Cdiff = add(C_b, neg(C_a))
    v3 = eq(add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
                  multiply(Cdiff, e)), T3)
    return v1, v2, v3

# --- Theorem 3: completeness ---
proof = cp_prove(rand(), rand())
v1, v2, v3 = cp_verify(*proof)
assert v1 and v2 and v3, "Theorem 3 FAILED"
print("Theorem 3 (completeness):          PASS")

# --- Theorem 4: extract witness from two transcripts with same (T1,T2,T3) ---
# We fix (k1, k2) to produce the same (T1, T2, T3), then force two
# different challenges by hashing with two different context strings
# (in production the different challenges arise from rewinding the R0).
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))

def responses(e):
    return (k1 - e * sk_alice) % ORDER, (k2 - e * r_b) % ORDER

e_1 = rand()
e_2 = rand()
while e_1 == e_2:
    e_2 = rand()
s1_a, s2_a = responses(e_1)
s1_b, s2_b = responses(e_2)

def inv_mod(a, p):
    return pow(a, -1, p)

delta_e = (e_1 - e_2) % ORDER
delta_e_inv = inv_mod(delta_e, ORDER)
sk_star = ((s1_b - s1_a) * delta_e_inv) % ORDER # should equal sk_alice
r_star = ((s2_b - s2_a) * delta_e_inv) % ORDER # should equal r_b

assert sk_star == sk_alice, "Theorem 4 sk extraction FAILED"
assert r_star == r_b, "Theorem 4 r extraction FAILED"
# Witnesses satisfy (S1), (S2), (S3):
assert eq(multiply(G1, sk_star), pk_alice)

```

```

assert eq(multiply(G1, r_star), R_b)
assert eq(add(C_a, neg(multiply(R_a, sk_star))),
          add(C_b, neg(multiply(pk_bob, r_star))))
print("Theorem 4 (special soundness):          PASS (sk, r extracted from two transcripts)")

# --- Theorem 5: simulator produces accepting transcript without witness ---
def cp_simulate(e):
    s1 = rand(); s2 = rand()
    T1 = add(multiply(G1, s1),      multiply(pk_alice, e))
    T2 = add(multiply(G1, s2),      multiply(R_b, e))
    Cdiff = add(C_b, neg(C_a))
    T3 = add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
              multiply(Cdiff, e))
    return (T1, T2, T3, e, s1, s2)

sim = cp_simulate(rand())
v1, v2, v3 = cp_verify(*sim)
assert v1 and v2 and v3, "Theorem 5 FAILED (simulated transcript rejected)"
print("Theorem 5 (HVZK):                      PASS (simulator accepts without witness)")

# --- Unsoundness counter-example: wrong M at destination ---
M_fake = multiply(G1, rand())
C_b_fake = add(M_fake, multiply(pk_bob, r_b))
# Honest prover tries to produce a proof for (E_alice, (R_b, C_b_fake))
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
tag = "|".join(str(v) for v in
               [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b_fake, T1, T2, T3])
e_f = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
s1_f = (k1 - e_f * sk_alice) % ORDER
s2_f = (k2 - e_f * r_b) % ORDER
v1_f = eq(add(multiply(G1, s1_f), multiply(pk_alice, e_f)), T1)
v2_f = eq(add(multiply(G1, s2_f), multiply(R_b, e_f)), T2)
Cdiff_f = add(C_b_fake, neg(C_a))
v3_f = eq(add(add(multiply(pk_bob, s2_f), neg(multiply(R_a, s1_f))),
              multiply(Cdiff_f, e_f)), T3)
assert v1_f and v2_f and not v3_f, "Unsoundness: fake M proof should fail at V3"
print("Counter-example (M != M'):             V1,V2 PASS; V3 FAIL (as required)")

Theorem 3 (completeness):          PASS
Theorem 4 (special soundness):     PASS (sk, r extracted from two transcripts)
Theorem 5 (HVZK):                  PASS (simulator accepts without witness)
Counter-example (M != M'):         V1,V2 PASS; V3 FAIL (as required)

```

How to read the output: Four PASS lines plus the counter-example. The counter-example is the mirror image of Theorem 4: when the statement is *false* (different message points), the first two verification equations still pass (Alice's key and randomness are genuine) but (V3) – the "same-message" equation – fails. This is precisely the mathematical impossibility the prose in the Identity document refers to when it says "no valid proof exists."

4 Part III: End-to-End Re-Encryption Correctness

4.1 Chain of Trust

The on-chain identity pipeline chains together the two primitives proved above:

KYC ceremony:	issuer signs m	-->	σ on m	(q-SDH)
Fountain:	rerandomize σ	-->	σ' on same m	(Theorem 1, 2)
Registration:	PS + NIZK check	-->	E_{alice} bound to same m	(PS verify + NIZK soundness)
approve(Bob):	Alice re-encrypts	-->	E_{bob}	(Theorem 3)
	Chaum-Pedersen	-->	verified on chain	(Theorems 3, 4, 5)
Decryption:	Bob computes M	-->	$M = C_b - sk_{\text{bob}} \cdot R_b$	(ElGamal correctness)
Two-channel:	Bob receives	-->	$H(\text{identity_data}) \cdot G = M$	(hash commitment)

4.2 Theorem 6 (End-to-End Re-Encryption Correctness)

Statement. Suppose the BUCK contract:

1. holds a registered credential $E_{\text{alice}} = (R_a, C_a)$ together with pk_{alice} and a PS signature σ' that verified at registration time against a trusted issuer public key (X, Y) ;
2. accepts a Chaum-Pedersen proof π supplied by `msg.sender` = Alice, with spender pk_{bob} and submitted ciphertext $E_{\text{bob}} = (R_b, C_b)$.

Then, under the DLog assumption on G_1 and in the ROM, there exists $M = m \cdot G$ such that

$$C_a - sk_{\text{alice}} \cdot R_a = M = C_b - sk_{\text{bob}} \cdot R_b,$$

and the scalar m is the issuer-certified identity scalar covered by σ' .

Proof sketch. The registration check evaluates $e(\sigma'_1, X + mY) = e(\sigma'_2, g_2)$ together with a NIZK that binds the same m to E_{alice} (so $C_a - sk_{\text{alice}} R_a = m \cdot G$); existential unforgeability of PS under q-SDH ensures m is the scalar the issuer signed. The contract reads E_{alice} from storage, not calldata, so the value is frozen.

Theorem 4 (special soundness) plus the forking lemma reduction to DLog gives: from any adversary producing an accepting π with non-negligible probability there is an extractor outputting witnesses (sk^*, r^*) with $C_a - sk^* R_a = C_b - r^* pk_{\text{bob}}$. Since $pk_{\text{alice}} = sk^* \cdot G = sk_{\text{alice}} \cdot G$ and G is a generator, $sk^* = sk_{\text{alice}}$; so both sides equal the registered M . Therefore E_{bob} decrypts under sk_{bob} to the same M that the issuer's PS signature attests to. ■

4.3 Why This Suffices for Every Downstream Use Case

- **approve / transfer (Identity):** Bob decrypts E_{bob} and obtains M ; combined with off-chain `identity_data` delivery and the check $H(\text{identity_data}) \cdot G = M$, he learns Alice's real identity with cryptographic assurance.
- **A-spend predicate (Notes):** The recipient-to-depositor binding in addressed Notes (flavors A1 and A2) is the same Chaum-Pedersen statement, lifted inside a SNARK. Theorem 4's soundness lifts directly: a SNARK-verified instance of the Chaum-Pedersen relation implies the same extractable witness relation, so the mint-time recipient credential E_{note} and the spend-time depositor credential $E_{\text{dep}}^{(\text{reg})}$ share M (formalized as Theorem 8 in Part IV). Identity laundering through the note pool is therefore infeasible.

- **Identity Guardians (approveFor, recovery, cosigning):** The Schnorr proof of knowledge of sk is a one-statement specialization of the above (only (S1) with a Fiat-Shamir binding to $(\text{msg.sender}, \text{principal})$). Theorems 3-5 apply verbatim with the second and third statements dropped.

DRAFT

5 Part IV: BUCK Notes Spend Predicates (A1, A2, B1)

This Part proves correctness of the spend predicates underlying BUCK Notes (Notes). A note is an asynchronous *deferred approve* handshake mediated by a global commitment pool: the issuer’s half is encoded in a Merkle commitment at mint time, the depositor’s half is supplied at spend time inside a zk-SNARK. The three valid *flavors* – **A1** (addressed cheque, public issuer), **A2** (addressed cheque, private issuer), and **B1** (bearer cheque, public issuer) – distinguish themselves by the commitment payload but collapse to two spend-circuit shapes:

- **A-spend** (covers A1 and A2): the depositor proves Chaum-Pedersen equality (Theorem 6) between the recipient-binding ciphertext E_{note} baked into the commitment and the depositor’s *current* registered credential, without revealing the encrypted identity scalar m .
- **B-spend** (covers B1): the depositor proves knowledge of the bearer secret ρ , proves knowledge of their registered sk_{dep} via Schnorr PoK, and verifies a Schnorr signature attesting the public issuer’s identity.

Throughout, the SNARK proof system (Groth16, PLONK, or any knowledge-sound zk-SNARK over BN254) is treated as a black box. *Soundness of the SNARK proof system itself is not analyzed here*; we only show that the in-circuit statements – assuming standard SNARK extraction – imply the desired cryptographic guarantees. Soundness of the underlying sigma-protocol relations (CP-equality and Schnorr) is established in Parts I-III; this Part lifts those results to the SNARK setting and adds the new pieces specific to the pool architecture: mint re-randomization correctness (Theorem 7), spend-predicate soundness (Theorems 8, 9), nullifier construction (Theorem 10), and preservation of the BUCK mutual decryptability invariant (Theorem 11).

5.1 Setup

Pool state. The BUCK contract maintains

- **noteTree**: an append-only Poseidon Merkle tree of fixed depth 20 ($\max 2^{20} \approx 10^6$ outstanding commitments) with leaves

$$cm = \text{Poseidon}_5(\text{flavor}, v, \rho, \text{idHash}, \text{predicate}),$$

where idHash is a Poseidon digest of the per-flavor payload (the recipient ciphertext E_{note} for A-spends, the issuer-key bundle for B-spends). Equivalently, writing $H_{\text{cm}}(\text{flavor}, v, \rho, \text{payload}, \text{predicate}) := \text{Poseidon}_5(\text{flavor}, v, \rho, H_{\text{payload}}(\text{payload}), \text{predicate})$. Internal nodes use Poseidon-2 over BN254’s scalar field; empty leaves hash to a fixed domain-separated **ZERO_VALUE**.

- **nullifiers**: a set of spent nullifiers.
- **roots**: a ring buffer of recent **noteRoot** values (window 30) that spend proofs may pin against.

Mint architecture (Phase 7-bis batch mint). `Notes.mint` no longer touches the Merkle tree directly. Instead, each mint submits a Groth16 proof of in-circuit incremental insertion of N fresh leaves – N is fixed per pinned verifier (currently $N \in \{16, 32\}$ on L1; see alberta-buck-notes-rollup-mint.org for the per- N dispatch architecture and gas-economics rationale). The contract verifies the proof, asserts **oldRoot** matches the live root and **nextLeafIndex** matches the live size, accepts the SNARK-attested **newRoot** into the root-history ring buffer, advances **nextLeafIndex**

by N , and pulls **totalFace** BUCK from the issuer. Per-leaf range checks ($v_i, \text{totalFace} \in [0, 2^{128})$) and value conservation ($\sum_i v_i == \text{totalFace}$) are enforced inside the same circuit.

The pool's BUCK balance and accumulated BUCK-age are tracked by the standard ERC-20 ledger; demurrage arithmetic is orthogonal to the cryptographic predicate and is not analyzed here.

Identity primitives (recap). Each registered identity has

- an issuer-certified scalar $m \in \mathbb{Z}_q$ and identity point $M = m \cdot G \in G_1$ (PS-signed at registration; see Part I);
- an ElGamal credential $E^{(\text{reg})} = (R^{\text{reg}}, C^{\text{reg}}) = (rG, M + r \cdot pk)$ on file in IdentityRegistry, where $pk = sk \cdot G$.

Identity Fountain re-issuance gives a *fresh* $(sk', pk', E^{(\text{reg})'})$ bound to the same M .

Mint construction. Let $E_r = (R_r, C_r) = (r_r G, M_{\text{rec}} + r_r \cdot pk_{\text{rec}})$ be the recipient's registered credential (read by the issuer at mint time). Let $r' \leftarrow \mathbb{Z}_q^*$ be fresh issuer-side randomness. Define

$$E_{\text{note}} := (R_r + r'G, C_r + r' \cdot pk_{\text{rec}}).$$

The mint payloads, by flavor, are:

Flavor	Payload
A2	$(E_{\text{note}}, E_{\text{iss-for-rec}})$
A1	$(E_{\text{note}}, m_{\text{iss}}, \sigma_{\text{iss}})$
B1	$(m_{\text{iss}}, \sigma_{\text{iss}}, pk_{\text{iss}})$

where $E_{\text{iss-for-rec}}$ is an ElGamal encryption of M_{iss} (and a means to recover pk_{iss} , either by carrying it alongside or by IdentityRegistry lookup of m_{iss}) under pk_{rec} , and σ_{iss} is a Schnorr signature on cm under $pk_{\text{iss}} = m_{\text{iss}} \cdot G$.

Nullifier construction. With domain separators $\{A, B\}$ and Poseidon-modelled-as-RO H_{nf} :

$$nf^{(A)} = H_{\text{nf}}(A \parallel M_{\text{rec}} \parallel \rho), \quad nf^{(B)} = H_{\text{nf}}(B \parallel \rho).$$

Design note. An earlier draft of **alberta-buck-notes.org** keyed the A-spend nullifier on sk_{dep} rather than on M_{rec} . Keying on M_{rec} – the issuer-certified identity point, recovered in-circuit by ElGamal decryption of E_{note} – keeps the nullifier independent of *which* 'sk' value was used to decrypt (a privacy improvement over sk-keyed nullifiers) and prevents a recipient who somehow holds two simultaneously-valid credentials for the same M from double-depositing. **It does not provide key-loss recovery.** The spend-time decryption step still requires the specific sk_{rec} paired with the mint-time pk_{rec} ; if that key is lost, the note is unspendable regardless of Identity re-issuance. The notes document has been updated to match; see Theorem 8 below for the formal statement and Section "Key-Pair Binding and Loss Semantics" in **alberta-buck-notes.org** for the operational implications.

5.2 Theorem 7 (Mint Re-randomization Correctness)

Statement. Let $E_r = (R_r, C_r) = (r_r G, M_{\text{rec}} + r_r \cdot pk_{\text{rec}})$ be a valid ElGamal ciphertext of M_{rec} under pk_{rec} , and let $r' \in \mathbb{Z}_q$ be arbitrary. Then $E_{\text{note}} := (R_r + r'G, C_r + r' \cdot pk_{\text{rec}})$ is a valid ElGamal ciphertext of the same M_{rec} under the same pk_{rec} , with effective randomness $r_r + r'$.

Proof. By group arithmetic on G_1 :

$$\begin{aligned} R_r + r'G &= r_r G + r'G = (r_r + r')G, \\ C_r + r' \cdot pk_{\text{rec}} &= M_{\text{rec}} + r_r \cdot pk_{\text{rec}} + r' \cdot pk_{\text{rec}} \\ &= M_{\text{rec}} + (r_r + r') \cdot pk_{\text{rec}}. \end{aligned}$$

The pair has the canonical ElGamal form $(\rho G, M_{\text{rec}} + \rho \cdot pk_{\text{rec}})$ with $\rho = r_r + r'$. ■

Information-theoretic remarks.

1. **Issuer ignorance.** The issuer manipulates E_r as group elements only – she never decrypts. She therefore learns neither M_{rec} , m_{rec} , nor r_r . This is load-bearing for Theorem 10 (issuer-cannot-precompute the A-spend nullifier) and for the BUCK privacy story (an issuer cannot enumerate registered identities by re-randomizing their credentials).
2. **Distribution.** If $r' \leftarrow \mathbb{Z}_q^*$ is uniform, then $r_r + r'$ is uniform on $\mathbb{Z}_q$ (translation by a fixed value). E_{note} is therefore distributed identically to a fresh ElGamal encryption of M_{rec} under pk_{rec} – the same information-theoretic argument as Theorem 2’s signature rerandomization, applied to the ciphertext. Two independently-minted A-notes for the same recipient are unlinkable to anyone who does not hold sk_{rec} .

5.3 Theorem 8 (A-Spend Soundness)

Statement. Suppose the BUCK contract accepts a SNARK proof $\pi^{(A)}$ for the A-spend circuit (alberta-buck-notes.org, "A-Spend Circuit") on public inputs (`noteRoot`, `nf`, `v`, `depositorRegistryEntry`). Then under (i) knowledge soundness of the SNARK system, (ii) the DLog assumption on G_1 , and (iii) the ROM, there exists an extractor $\mathcal{E}$ that outputs a witness (flavor, v , ρ , payload, path, sk_{dep}) such that:

1. $cm = H_{\text{cm}}(\text{flavor}, v, \rho, \text{payload})$ lies on the Merkle path `path` under `noteRoot`.
2. The extracted sk_{dep} satisfies $pk_{\text{dep}} = sk_{\text{dep}} \cdot G$, where pk_{dep} is registered in `IdentityRegistry` under `depositorRegistryEntry`.
3. `payload` contains a ciphertext $E_{\text{note}} = (R_n, C_n)$, and the depositor’s identity point $M := C^{\text{reg}} - sk_{\text{dep}} \cdot R^{\text{reg}}$ (decrypted from their registered credential) satisfies $C_n - sk_{\text{dep}} \cdot R_n = M$.
4. $nf = H_{\text{nf}}(A \parallel M \parallel \rho)$.
5. If flavor = A1: the payload also contains $(m_{\text{iss}}, \sigma_{\text{iss}})$ with σ_{iss} verifying as a Schnorr signature on cm under $pk_{\text{iss}} := m_{\text{iss}} \cdot G$.

Proof. Knowledge soundness of the SNARK gives an extractor producing witnesses satisfying every constraint of the circuit (with overwhelming probability). Constraints 1, 2, 4, and 5 are direct algebraic identities on the extracted witness.

Constraint 3 is the in-circuit Chaum-Pedersen equality

$$C_n - sk_{\text{dep}} \cdot R_n = C^{\text{reg}} - sk_{\text{dep}} \cdot R^{\text{reg}}.$$

Theorem 4 (special soundness of Chaum-Pedersen) establishes that any accepting CP-equality statement implies the two ciphertexts share the same encrypted message point under their respective public keys. In the present case both are encrypted under pk_{dep} (the depositor’s registered key), so equality of decrypted plaintexts reduces to equality in G_1 – precisely constraint 3.

By the registration-time PS-signature binding (Theorem 6, chain of trust), M is the issuer-certified identity point covered by the depositor’s PS signature at `IdentityRegistry` registration. By Theorem 7, E_{note} was constructed at mint as a re-randomization of the recipient’s registered credential and therefore encrypts M_{rec} . CP-equality forces $M = M_{\text{rec}}$. ■

Implications.

- **Wrong identity rejected.** No prover lacking knowledge of an sk_{dep} bound (by registration) to the same m_{rec} chosen at mint can produce an accepting $\pi^{(A)}$. An adversary holding a credential bound to $M' \neq M_{\text{rec}}$ fails CP-equality: their $M = M' \neq M_{\text{rec}}$.
- **Issuer cannot self-redeem an A-note.** The issuer knows pk_{rec} and r' but not sk_{rec} ; she cannot decrypt E_{note} to verify M_{rec} and cannot supply an sk_{dep} bound to that M_{rec} unless she *is* the recipient (degenerate "self-wormhole" subcase where issuer $\equiv$ recipient).
- **A-notes are bound to the mint-time key pair.** The circuit uses a *single* witness sk_{dep} to decrypt both the note's $E_{\text{note}} = (R_n, C_n)$ (encrypted under pk_{rec} at mint time) and the depositor's currently-registered $E^{(\text{reg})}$ (encrypted under the depositor's current pk_{dep}). Constraint 3 therefore forces $pk_{\text{rec}} = pk_{\text{dep}}$: the same public key must appear on both ciphertexts, otherwise $C_n - sk_{\text{dep}} \cdot R_n = M_{\text{rec}} + (r_r + r') \cdot (pk_{\text{rec}} - pk_{\text{dep}}) \neq M_{\text{rec}}$. Identity re-issuance produces a fresh (sk', pk') bound to the same M but revokes the previous credential (**identity.org**, Attack 7); the old IdentityRegistry entry fails **isVerified** and cannot be used, while the new entry has $pk' \neq pk_{\text{rec}}$ and fails CP-equality. Consequently, **loss of sk_{rec} renders the A-note unspendable**. Key-management practice for A-note recipients (especially long-hold A2 cold-storage instruments) must treat sk_{rec} as load-bearing for note recoverability.

5.4 Theorem 9 (B-Spend Soundness)

Statement. Suppose the BUCK contract accepts a SNARK proof $\pi^{(B)}$ for the B-spend circuit on public inputs (**noteRoot**, nf , v , **issuerPkClaim**, **depositorRegistryEntry**). Then under knowledge soundness of the SNARK, the DLog assumption on G_1 , the EUF-CMA security of Schnorr signatures under DLog and the ROM⁴, and the ROM, there exists an extractor outputting $(v, \rho, \text{payload}, \text{path}, sk_{\text{dep}})$ such that:

1. $cm = H_{\text{cm}}(B1, v, \rho, \text{payload})$ lies on **path** under **noteRoot**.
2. $nf = H_{\text{nf}}(B \parallel \rho)$.
3. **payload** contains $(m_{\text{iss}}, \sigma_{\text{iss}})$, and σ_{iss} is a valid Schnorr signature on cm under $pk_{\text{iss}} := m_{\text{iss}} \cdot G$, with pk_{iss} registered in IdentityRegistry.
4. $pk_{\text{dep}} = sk_{\text{dep}} \cdot G$ is registered in IdentityRegistry under **depositorRegistryEntry**.
5. $v = \text{faceValue}$.

Proof. The SNARK extractor produces witnesses satisfying every circuit constraint. Constraint 4 of the B-spend circuit (**alberta-buck-notes.org**, "B-Spend Circuit") is a Schnorr proof of knowledge of sk_{dep} for pk_{dep} – a one-statement specialization of Chaum-Pedersen (drop relations (S2), (S3) of Part II), inheriting Theorem 4's special-soundness extractor and Theorem 5's HVZK simulator. Constraint 3 is the verification of the Schnorr signature σ_{iss} on cm under pk_{iss} ; Schnorr signatures are EUF-CMA secure under DLog and the ROM⁴, so the signature could only have been produced by the holder of $sk_{\text{iss}} = m_{\text{iss}}$. Constraints 1, 2, 5 are direct algebraic identities. ■

Implications.

⁴Schnorr, C.-P. "Efficient Signature Generation by Smart Cards." Journal of Cryptology 4(3), 1991, pp. 161-174. The Schnorr signature scheme is EUF-CMA secure under the discrete logarithm assumption in the random oracle model; see Pointcheval and Stern (2000) ² for the forking-lemma reduction, and Bellare and Neven, "Multi-signatures in the Plain Public-Key Model" (CCS 2006) for tightened security bounds.

- **Registered depositor.** Every B-spend deposit attaches a registered Identity to the deposit transaction, satisfying the BUCK compliance hook (KYC-attached identity at every transfer endpoint).
- **Public issuer attestation.** The issuer of a B1 note is publicly attested by m_{iss} and the EUF-CMA-secure signature; downstream observers and regulators can audit which public issuer printed the note.
- **Knowledge of ρ is the bearer property.** The predicate is satisfied by *any* registered identity that knows ρ . This is exactly the cashier’s-cheque semantics: possession of the opening conveys spend authority.

Remark on issuer race. The issuer who chose ρ at mint trivially knows it and can race the bearer to deposit – this is a *civil-trust* property, not a cryptographic vulnerability. The deposit event carries $E_{\text{dep-for-iss}}$, so an issuer who steals their own bearer cheque is identified at the moment of theft (Theorem 11). The cryptography places no prior restraint on the issuer; the legal and reputational system does. The same trust model applies to a paper cashier’s cheque, where the issuing bank could in principle redirect the draft.

5.5 Theorem 10 (Nullifier Construction and Double-Spend Prevention)

Statement (A-spend nullifier, keyed by M_{rec}).

1. **Determinism.** $nf^{(A)} = H_{\text{nf}}(A \parallel M_{\text{rec}} \parallel \rho)$ depends on the depositor’s witness only through (M_{rec}, ρ) . Keying on the identity point rather than on sk_{dep} (a) keeps the nullifier value independent of which specific signing key was used to decrypt E_{note} – a modest privacy improvement – and (b) prevents any pathological scenario in which a recipient holds multiple simultaneously-valid credentials for the same M from yielding distinct nullifiers and enabling double-deposit.
2. **Issuer non-computability (in the ROM).** The issuer holds pk_{rec}, r_r, r' and ρ , but not sk_{rec} . Computing $nf^{(A)}$ requires $M_{\text{rec}} = C_r - r_r \cdot pk_{\text{rec}}$ (equivalent to ElGamal-decrypting E_r without sk_{rec}) – breaking ElGamal IND-CPA, which reduces to DLog on pk_{rec} . Hence the issuer cannot precompute the nullifier and observe its on-chain appearance to detect a spend.
3. **Recipient computability (key-dependent).** The recipient decrypts E_{note} with the specific sk_{rec} paired with the mint-time pk_{rec} to recover M_{rec} , reads ρ from the (decrypted) commitment opening, and computes $nf^{(A)}$ – exactly the data the spend circuit requires. **Loss of sk_{rec} is unrecoverable:** no alternative credential can decrypt E_{note} , so the nullifier – and hence the spend – cannot be produced even if Identity re-issuance preserves M_{rec} on a fresh keypair. The nullifier-value invariance across re-issuance is a formal property of the hash function, not an operational recovery mechanism.

Statement (B-spend nullifier, keyed by ρ alone).

1. **Public derivability.** Any party knowing ρ can compute $nf^{(B)} = H_{\text{nf}}(B \parallel \rho)$. This is precisely the first-come-first-served property of bearer cheques: whoever publishes ρ first wins.

Statement (double-spend prevention, both shapes).

1. Two accepting spend proofs on the same cm by the same flavor produce the same nullifier (by 1 or by 4); the contract's check $nf \notin \text{nullifiers}$ rejects the second submission.

Statement (cross-flavor disjointness).

1. Domain separation (A vs. B prefix) makes $\{nf^{(A)}\} \cap \{nf^{(B)}\} = \emptyset$ except on a negligible RO-collision event. A single **nullifiers** set safely tracks both flavors.

Proof sketches. Determinism (1) and public derivability (4) are syntactic from the PRF definition. Issuer non-computability (2) is the IND-CPA reduction outlined above; making the random oracle programmable, an issuer who outputs the correct $nf^{(A)}$ on $(pk_{\text{rec}}, E_r, r', \rho)$ with non-negligible probability allows the simulator to read M_{rec} off the RO-query log and thereby decrypt E_r . Double-spend rejection (5) is a contract-level invariant on the **nullifiers** set. Cross-flavor disjointness (6) follows from RO collision resistance. ■

5.6 Theorem 11 (Mutual-Decryptability Preservation)

Statement. For every accepted spend (A1, A2, or B1):

1. The depositor learns the issuer's identity scalar m_{iss} (and thus M_{iss} , and via off-chain identity-data lookup the issuer's real-world identity).
2. The issuer learns the depositor's identity scalar m_{dep} (and thus M_{dep} , and via the same identity-data path the depositor's real-world identity).

This is the BUCK Identity invariant – every transfer's two counterparties mutually disclose – preserved under the Notes pool detour.

Proof (by cases).

- *A1, B1 (public issuer).* The mint payload contains m_{iss} in plaintext, witnessed by σ_{iss} under $pk_{\text{iss}} = m_{\text{iss}} \cdot G$. The depositor reads it directly. EUF-CMA security of Schnorr (Theorem 9 citation) guarantees the issuer claim cannot be forged.
- *A2 (private issuer).* The mint payload contains $E_{\text{iss-for-rec}}$, an ElGamal encryption of M_{iss} under pk_{rec} (with pk_{iss} similarly encrypted, or recovered from IdentityRegistry by lookup of m_{iss} once decrypted). The recipient decrypts using sk_{rec} by ElGamal correctness.
- *Depositor $\rightarrow$ issuer (all flavors).* The deposit transaction emits an event payload $E_{\text{dep-for-iss}}$, an ElGamal encryption of M_{dep} under pk_{iss} . The issuer decrypts using sk_{iss} . For A1 and B1 the issuer is public so pk_{iss} is directly known to the depositor; for A2 the depositor recovers pk_{iss} via the payload-decryption path above.

In each case both parties recover the other's identity point and – via the standard off-chain identity_data delivery path (Theorem 6) – the underlying real-world identity. The on-chain observer learns neither M_{iss} (for A2), M_{dep} , nor the linkage between cm and the deposit event without solving DLog on the relevant ElGamal keys or guessing ρ (a uniform q -element). ■

Privacy corollary. Bilateral disclosure between the two counterparties is preserved. The "mass-harvest" threat – bulk on-chain analytics, KYC-provider subpoena scope expansion, chain-analytics enrichment – is broken: the chain shows cm , nf , public flavor metadata, and the deposit event, but cannot link any pair without the corresponding counterparty secret. See alberta-buck-notes.org, "Privacy: Defeating Mass Harvest, Preserving Bilateral Disclosure," for the threat-model framing.

5.7 Sanity Check (py_ecc on BN254)

```
# Reuse ORDER, G1, multiply, add, neg, eq, rand, hashlib, m, sk_alice,
# pk_alice, sk_bob, pk_bob from Parts I-III. Issuer = Alice; Bob is
# the recipient, with a fresh registered identity scalar m_rec.

m_rec = int(hashlib.sha256(b"Bob Recipient | Alberta | AIC-2026").hexdigest(),
              16) % ORDER
M_rec = multiply(G1, m_rec)
r_r = rand()
R_r = multiply(G1, r_r)
C_r = add(M_rec, multiply(pk_bob, r_r)) # Bob's ElGamal credential

def elgamal_decrypt(R, C, sk):
    return add(C, neg(multiply(R, sk)))

assert eq(elgamal_decrypt(R_r, C_r, sk_bob), M_rec)

# --- Theorem 7: mint re-randomization correctness (A2/A1) ---
r_prime = rand()
R_n = add(R_r, multiply(G1, r_prime))
C_n = add(C_r, multiply(pk_bob, r_prime))
assert eq(elgamal_decrypt(R_n, C_n, sk_bob), M_rec), \
    "Theorem 7 FAILED: E_note must decrypt to M_rec"
print("Theorem 7 (mint re-randomization): PASS (E_note -> M_rec)")

# --- Theorem 8: A-spend CP-equality holds for the recipient ---
def cp_a_check(R_n, C_n, R_reg, C_reg, sk_dep):
    return eq(elgamal_decrypt(R_n, C_n, sk_dep),
              elgamal_decrypt(R_reg, C_reg, sk_dep))
assert cp_a_check(R_n, C_n, R_r, C_r, sk_bob)
print("Theorem 8 (A-spend, recipient deposits): PASS (CP M_note=M_rec)")

# --- Theorem 8 (negative): issuer Alice cannot self-redeem ---
M_iss = multiply(G1, m) # Alice's identity point
M_alice_decrypts_note = elgamal_decrypt(R_n, C_n, sk_alice)
assert not eq(M_alice_decrypts_note, M_iss), \
    "Issuer self-redeem must fail: decrypt(E_note, sk_alice) != M_iss"
print("Theorem 8 (issuer cannot redeem own A-note): PASS (CP fails)")

# --- Theorem 10.1: nullifier determinism and key-pair binding ---
def H_nf(domain, *fields):
    blob = domain + b"|" + b"|".join(str(f).encode() for f in fields)
    return int(hashlib.sha256(blob).hexdigest(), 16) % ORDER

rho = rand()
# Recipient with mint-time sk_rec decrypts E_note -> M_rec and
# computes the nullifier. This is the only path that works.
M_rec_decrypted = elgamal_decrypt(R_n, C_n, sk_bob)
assert eq(M_rec_decrypted, M_rec), \
    "Theorem 10.1 FAILED: sk_rec must decrypt E_note to M_rec"
nf_A = H_nf(b"A", M_rec_decrypted, rho)
# Nullifier value is a pure function of (M_rec, rho): recomputing
# with the same inputs yields the same value regardless of which
# credential is held. This is a hash-determinism property.
assert nf_A == H_nf(b"A", M_rec, rho), \
    "Theorem 10.1 FAILED: nullifier must be deterministic in (M_rec, rho)"
print("Theorem 10.1a (nullifier determinism in (M_rec, rho)): PASS")

# --- Theorem 10.1 (negative): re-issued recipient without sk_rec ---
# Bob re-issues (fresh sk_bob_2, pk_bob_2) bound to the same M_rec.
# Bob no longer has sk_bob. He cannot decrypt E_note:
sk_bob_2 = rand()
pk_bob_2 = multiply(G1, sk_bob_2)
M_wrong = elgamal_decrypt(R_n, C_n, sk_bob_2)
assert not eq(M_wrong, M_rec), \
    "Theorem 10.1 FAILED: sk_bob_2 must NOT decrypt E_note to M_rec"
# With the wrong M, the nullifier he could compute differs from nf_A,
```

```

# but more importantly the A-spend circuit's CP-equality constraint
# (C_n - sk*R_n == C_reg - sk*R_reg) cannot be satisfied: E_note is
# under pk_bob (old), E_reg_2 is under pk_bob_2 (new), so no single
# sk witness decrypts both to the same point. The re-issued
# recipient cannot produce an accepting spend proof.
r_r_2 = rand()
R_r_2 = multiply(G1, r_r_2)
C_r_2 = add(M_rec, multiply(pk_bob_2, r_r_2))
# sk_bob_2 decrypts E_reg_2 to M_rec ...
assert eq(elgamal_decrypt(R_r_2, C_r_2, sk_bob_2), M_rec)
# ... but the same sk_bob_2 does NOT decrypt E_note to M_rec:
assert not eq(elgamal_decrypt(R_n, C_n, sk_bob_2),
              elgamal_decrypt(R_r_2, C_r_2, sk_bob_2)), \
              "Theorem 10.1 FAILED: CP-equality must fail under key rotation"
print("Theorem 10.1b (A-note unspendable after key loss):      PASS (CP fails)")

# --- Theorem 10.4: B-spend nullifier publicly derivable from rho ---
nf_B_holder = H_nf(b"B", rho)
nf_B_eve = H_nf(b"B", rho)          # Eve only needs rho
assert nf_B_holder == nf_B_eve
assert nf_A_pre != nf_B_holder      # cross-flavor disjoint
print("Theorem 10.4 (B-spend nullifier public from rho):      PASS")

# --- Theorem 11: mutual decryptability round-trip ---
# A2 mint payload: E_iss-for-rec encrypts M_iss under pk_bob.
r_iss_for_rec = rand()
E_iss_R = multiply(G1, r_iss_for_rec)
E_iss_C = add(M_iss, multiply(pk_bob, r_iss_for_rec))
assert eq(elgamal_decrypt(E_iss_R, E_iss_C, sk_bob), M_iss), \
        "Theorem 11 FAILED: recipient must recover M_iss"
# Deposit-event payload: E_dep-for-iss encrypts M_dep under pk_alice.
M_dep = M_rec                      # depositor = recipient
r_dep_for_iss = rand()
E_dep_R = multiply(G1, r_dep_for_iss)
E_dep_C = add(M_dep, multiply(pk_alice, r_dep_for_iss))
assert eq(elgamal_decrypt(E_dep_R, E_dep_C, sk_alice), M_dep), \
        "Theorem 11 FAILED: issuer must recover M_dep"
print("Theorem 11 (mutual decryptability round-trip):          PASS")

Theorem 7  (mint re-randomization):          PASS (E_note -> M_rec)
Theorem 8  (A-spend, recipient deposits):    PASS (CP M_note=M_reg)
Theorem 8  (issuer cannot redeem own A-note): PASS (CP fails)
Theorem 10.1a (nullifier determinism in (M_rec,rho)): PASS
Theorem 10.1b (A-note unspendable after key loss): PASS (CP fails)
Theorem 10.4 (B-spend nullifier public from rho): PASS
Theorem 11 (mutual decryptability round-trip): PASS

```

How to read the output: Seven PASS lines. Theorem 7 confirms the mint construction yields a valid encryption of the recipient's M under their own pk . The two Theorem 8 lines show the recipient succeeding (positive case) and the issuer failing (negative case): only credentials bound to the same M as the note can produce a CP-equality witness. Theorem 10.1a is nullifier-value determinism (a pure hash property: same inputs give same output). Theorem 10.1b is the key-pair-binding check: a re-issued recipient holding a fresh sk' (but lacking the mint-time sk_{rec}) cannot satisfy CP-equality, so the note is unspendable – the honest converse of the superseded "key-loss recovery" framing. Theorem 10.4 demonstrates the public derivability of the B-spend nullifier (anyone who knows ρ can compute it – the bearer-cheque property). Theorem 11 closes the mutual-decryptability loop: issuer-side payload encrypts M_{iss} for the recipient, deposit-side payload encrypts M_{dep} for the issuer – both decrypt correctly under the holder's sk .

5.8 Worked Example: End-to-End A1 Note Transfer (SNARK Relations)

The block above asserts each theorem in isolation. Below is a single end-to-end run of one A1 note (addressed cheque, public issuer) from issuance to deposit, structured as the two SNARK relations the prover would attest to and the contract would verify – `mint_relation` and `spend_relation` – plus a toy Merkle tree and nullifier set to make the contract’s role explicit. An A2 transfer is the same relation with the issuer-signature constraint dropped; a B1 transfer drops the CP-equality constraint and re-keys the nullifier on ρ instead of M_{rec} .

```
# Reuses ORDER, G1, multiply, add, neg, eq, rand, hashlib, m, sk_alice,
# pk_alice, sk_bob, pk_bob, M_rec, R_r, C_r from prior blocks.

def H_cm(*fields):
    blob = b"buck-cm|" + b"|".join(str(f).encode() for f in fields)
    return int(hashlib.sha256(blob).hexdigest(), 16) % ORDER

def H_node(l, r):
    return int(hashlib.sha256(b"merk|" + str(l).encode() + b"|"+
                             str(r).encode()).hexdigest(), 16) % ORDER

def H_e(*fields):
    return int(hashlib.sha256(b"|".join(str(f).encode()
                                         for f in fields)).hexdigest(), 16) % ORDER

def schnorr_sign(sk, msg):
    k = rand(); R = multiply(G1, k)
    e = H_e(R, multiply(G1, sk), msg)
    return (R, (k - e * sk) % ORDER)

def schnorr_verify(pk, msg, sig):
    R, s = sig
    e = H_e(R, pk, msg)
    return eq(add(multiply(G1, s), multiply(pk, e)), R)

def merkle(leaves, depth=3):
    # 8-leaf toy tree
    layer = list(leaves) + [0] * (2**depth - len(leaves))
    layers = [layer]
    while len(layers[-1]) > 1:
        prev = layers[-1]
        layers.append([H_node(prev[2*i], prev[2*i+1])
                       for i in range(len(prev)//2)])
    return layers

def merkle_path(layers, idx):
    path = []
    for layer in layers[:-1]:
        path.append((idx & 1, layer[idx ^ 1])); idx >>= 1
    return path

def merkle_root(leaf, path):
    h = leaf
    for bit, sib in path:
        h = H_node(sib, h) if bit else H_node(h, sib)
    return h

# --- MINT (A1): Alice issues a 1,000-BUCK note to Bob -----
face = 1000
m_iss_pub = m                                # A1: issuer M is in the clear
r_prime = rand()
R_n = add(R_r, multiply(G1, r_prime))         # E_note = re-randomized E_r
C_n = add(C_r, multiply(pk_bob, r_prime))
rho = rand()
cm = H_cm("A1", face, rho, R_n, C_n, m_iss_pub)
sig_iss = schnorr_sign(m_iss_pub, cm)         # A1: sk = m_iss, pk = m_iss * G1

def mint_relation_A1(pub, wit):
    # what the Mint SNARK proves
```

```

    face_p, cm_p = pub
    R_r_w, C_r_w, pk_rec_w, r_prime_w, rho_w, m_iss_w, sig_w = wit
    R_n_w = add(R_r_w, multiply(G1, r_prime_w))
    C_n_w = add(C_r_w, multiply(pk_rec_w, r_prime_w))
    if H_cm("A1", face_p, rho_w, R_n_w, C_n_w, m_iss_w) != cm_p:
        return False
    return schnorr_verify(multiply(G1, m_iss_w), cm_p, sig_w)

assert mint_relation_A1(
    (face, cm),
    (R_r, C_r, pk_bob, r_prime, rho, m_iss_pub, sig_iss))
print("Mint relation (A1):", PASS (cm opens; issuer sig verifies))

# Contract appends cm to noteTree; we put it at slot 3.
leaves = [0]*8; leaves[3] = cm
tree = merkle(leaves, depth=3)
root = tree[-1][0]
path = merkle_path(tree, 3)

# --- SPEND (A-spend): Bob deposits the note -----
sk_dep = sk_bob
R_reg, C_reg = R_r, C_r # Bob's currently registered cred.
M_rec_w = add(C_n, neg(multiply(R_n, sk_dep)))
nf = int(hashlib.sha256(b"buck-nf|A|" + str(M_rec_w).encode()
    + b"|" + str(rho).encode()).hexdigest(), 16) % ORDER

def spend_relation_A1(pub, wit): # what the A-Spend SNARK proves
    root_p, nf_p, face_p = pub
    (cm_w, rho_w, R_n_w, C_n_w, m_iss_w, sig_w, path_w,
     sk_dep_w, R_reg_w, C_reg_w) = wit
    # (1) Merkle inclusion of cm in noteTree
    if merkle_root(cm_w, path_w) != root_p: return False
    # (2) commitment opening binds (face, rho, E_note, m_iss)
    if H_cm("A1", face_p, rho_w, R_n_w, C_n_w, m_iss_w) != cm_w: return False
    # (3) nullifier keyed on M_rec recovered in-circuit
    M_rec_c = add(C_n_w, neg(multiply(R_n_w, sk_dep_w)))
    nf_c = int(hashlib.sha256(b"buck-nf|A|" + str(M_rec_c).encode()
        + b"|" + str(rho_w).encode()).hexdigest(), 16) % ORDER
    if nf_c != nf_p: return False
    # (4) CP-equality: registered credential decrypts to the same M
    M_reg_c = add(C_reg_w, neg(multiply(R_reg_w, sk_dep_w)))
    if not eq(M_rec_c, M_reg_c): return False
    # (5) A1-only: issuer signature on cm verifies under m_iss * G
    return schnorr_verify(multiply(G1, m_iss_w), cm_w, sig_w)

pub_spend = (root, nf, face)
wit_bob = (cm, rho, R_n, C_n, m_iss_pub, sig_iss, path,
    sk_dep, R_reg, C_reg)
assert spend_relation_A1(pub_spend, wit_bob)
print("Spend relation (A1, recipient):", PASS (Merkle + opening + nf + CP + sig))

# Negative: issuer Alice tries to redeem the note Bob is addressed on.
wit_alice = (cm, rho, R_n, C_n, m_iss_pub, sig_iss, path,
    sk_alice, R_reg, C_reg)
assert not spend_relation_A1(pub_spend, wit_alice)
print("Spend relation (issuer self-redeem):", PASS (rejected: CP-equality fails))

# --- Contract: nullifier set rejects double spend -----
spent = set()
def deposit(pub, wit):
    if not spend_relation_A1(pub, wit): return "reject:proof"
    if pub[1] in spent: return "reject:double-spend"
    spent.add(pub[1]); return f"accept:{pub[2]}"

assert deposit(pub_spend, wit_bob) == f"accept:{face}"
assert deposit(pub_spend, wit_bob) == "reject:double-spend"
print("Contract (nullifier set rejects replay):", PASS)

```

Mint relation (A1):	PASS (cm opens; issuer sig verifies)
Spend relation (A1, recipient):	PASS (Merkle + opening + nf + CP + sig)
Spend relation (issuer self-redeem):	PASS (rejected: CP-equality fails)
Contract (nullifier set rejects replay):	PASS

How to read the output: The two relations `mint_relation_A1` and `spend_relation_A1` are the precise statements a SNARK proof system (Groth16 / PLONK on BN254) would attest to in production – they are written as plain Python predicates here so the reader can trace each constraint by hand. Mint binds cm to $(face, \rho, E_{note}, m_{iss})$ and to the issuer’s Schnorr signature; spend re-derives cm from the witness, checks Merkle inclusion against the public root, recovers M_{rec} from E_{note} under sk_{dep} , checks the nullifier against $H_{nf}(A || M_{rec} || \rho)$, proves CP-equality with the depositor’s registered credential, and (A1 only) verifies the issuer signature on cm . The two negative cases – issuer-self-redeem and double-spend – show the relation rejecting the only two attacks the SNARK is responsible for filtering at this layer.

5.9 Mint-Circuit Status: Phase 7-bis Closure of the Earlier Open Questions

An earlier draft of this document (v0.4) flagged three open correctness obligations against the Phase-6 placeholder mint circuit (`circuits/mint.circom`). The Phase 7-bis batch-mint pivot (`circuits/mint_batch.circom`, deployed as per-N pinned variants `mint_batch_n${N}.circom`; see alberta-buck-notes-rollup-mint.org) closes all three at the mint layer. This section records the closure for the proof record so a reader cross-checking the prior draft can see which obligations are now discharged in the circuit vs. deferred to the spend circuit.

5.9.1 Closure 1 (Field-Overflow / Value-Conservation Range Check) – *Discharged in circuit*

The Phase-6 draft’s value-conservation constraint $\sum_{i=1}^N v_i === \text{totalFace}$ is an equation in $\mathbb{F}_r$ with $r \approx 2^{254}$ and therefore admits integer-overflow witnesses unless each v_i is bounded. `mint_batch.circom` now applies `Num2Bits(128)` to every v_i and to `totalFace` (constraint family (R) in the circuit source). With $N \leq 2^{20}$ (the tree-depth cap) and 128-bit per-leaf face, the integer sum cannot overflow: $N \cdot 2^{128} \leq 2^{148} \ll 2^{254} = r$. No spend circuit can be tricked into releasing more BUCK than was escrowed by an integer-vs-field-arithmetic mismatch on the mint side.

5.9.2 Closure 2 (Single `transferFrom` $\leftrightarrow$ N Commitments) – *Discharged by SNARK binding*

Phase 7-bis makes the `totalFace` public input of the mint SNARK both (i) the value passed to `buck.transferFrom` and (ii) the value the circuit forces to equal $\sum_i v_i$. The single `Transfer` event of size `totalFace` is therefore cryptographically bound to the Groth16 proof’s public inputs – which include the entire `cms[]` array on the per-N pinned verifiers (see alberta-buck-notes-rollup-mint.org, "Calldata delivery"). No separate batch-identifier storage slot is needed for soundness; on-chain analytics that want a `transfer` $\leftrightarrow$ batch correspondence can recover it from the `Minted` event’s `cms` parameter and the simultaneous `Transfer`.

5.9.3 Closure 3 (Spend-Circuit Trio) – *Mint half discharged; nullifier + spend-side value conservation deferred to spend.circom*

The earlier draft’s three coupled obligations – Merkle membership, nullifier determinism, and value conservation – now split cleanly between the two circuits:

1. **Merkle membership (mint side).** `mint_batch` performs an in-circuit dual Merkle walk per leaf (constraint family (M) in the circuit source): the empty-seat walk constrains every witness sibling against the rolling prior root, and the filled-seat walk advances the rolling root with cm_i inserted. After all N leaves the final rolling root is constrained to equal the public `newRoot`. The contract accepts `newRoot` into its root-history ring buffer; spend circuits prove inclusion against any root in that window. The Phase-2 flat `commitments[]` array is *gone*.
2. **Nullifier determinism (spend side).** The spend circuit must still recompute $nf = H_{\text{nf}}(\text{flavor}, M_{\text{rec}}, \rho)$ (A-spend) or $nf = H_{\text{nf}}(B, \rho)$ (B-spend) from the committed witness. This is the obligation Theorem 10 states; closure belongs to `circuits/spend.circom`.
3. **Value conservation (spend side).** The spend circuit must still output `faceReleased` equal to the committed v in the opening, with v the same 128-bit-bounded quantity range-checked at mint. The mint-side range check (Closure 1) remains load-bearing here: a spend circuit that trusts v without re-bounding it relies on the mint circuit having already done so.

The mint-circuit layout is now *frozen* at `Poseidon5(flavor, v,  $\rho$ , idHash, predicate)` over BN254 with tree depth 20; spend-circuit design no longer ratifies the mint-circuit layout because the layout is published and pinned by the per-N verifiers already deployed.

DRAFT

6 Security Assumptions Summary

Theorem	Primitive	Assumption needed	Conditional on crypto?
Theorem 1 (PS completeness)	PS	bilinearity of e	no (unconditional)
Theorem 2 (PS unlinkability)	PS	prime-order G_1 , uniform t	no (information-theoretic)
Theorem 3 (CP completeness)	CP	group axioms	no (unconditional)
Theorem 4 (CP special soundness)	CP	none for extractor; DLog for cheating-prover reduction	DLog (only for reduction)
Theorem 5 (CP HVZK)	CP	uniform k_1, k_2	no (perfect simulation)
NIZK corollary	CP + FS	ROM	ROM
Theorem 6 (end-to-end)	PS + CP	q-SDH (PS unforgeability), DLog, ROM	yes
Theorem 7 (mint re-randomization)	ElGamal	group axioms	no (unconditional)
Theorem 8 (A-spend soundness)	CP-in-SNARK	SNARK knowledge soundness, DLog, ROM	yes
Theorem 9 (B-spend soundness)	Schnorr	SNARK knowledge soundness,	yes
	+ Sig + PoK	Schnorr EUF-CMA (DLog + ROM)	
Theorem 10 (nullifier construction)	Poseidon-PRF	ROM (Poseidon), DLog (issuer-ignorance)	ROM, DLog
Theorem 11 (mutual decryptability)	ElGamal	ElGamal correctness; Schnorr EUF-CMA	DLog, ROM

Every assumption on the "yes" rows is already load-bearing for Ethereum itself (ECDSA on secp256k1 assumes DLog; every pairing-based SNARK verifier on L1 uses the same ROM reductions; every existing BLS-based rollup assumes q-SDH-class primitives). Alberta Buck introduces *no novel* cryptographic assumption.

7 Implementation Notes

7.1 What the Contract Must Enforce

The theorems assume a well-formed verifier. Three conditions on the on-chain code are load-bearing:

1. **Registration:** $\sigma'_1 \neq O$. Required by Theorem 1's non-degeneracy argument. A missing check admits the trivial signature attack (see Identity doc, Attack 9 / Link 1).
2. **Registration:** G_2 **subgroup check**. The `ecPairing` precompile (post-EIP-197) validates that X, Y lie in the correct prime-order subgroup of G_2 . Without this, small-subgroup attacks on the twist curve could forge pairings. Theorem 1 is stated over the correct subgroup; enforcement is the precompile's responsibility.
3. **Verification:** E_{alice} **read from storage, not calldata**. Theorem 6's reduction routes through the registration-time binding of E_{alice} to issuer-certified m . If Alice supplies E_{alice} via calldata the binding is severed and Theorem 6 does not apply.

7.2 What the Prover Must Enforce

- **Uniform randomness for t, k_1, k_2, r_b .** Theorems 2 and 5 depend on uniform distributions. A biased RNG converts statistical unlinkability into a detectable correlation and converts perfect HVZK into approximate HVZK with an adversarial advantage proportional to the bias.
- **Fiat-Shamir domain separation.** The hash input must include all public inputs and context (`msg.sender`, `spender`, `chainid`). Omitting any one opens a cross-context replay path; the forking lemma reduction is unaffected, but the NIZK is no longer bound to the intended transaction.

7.3 Scope Not Covered Here

- *Knowledge soundness* of the registration-time NIZK that binds σ' to E_{alice} : this is a standard Schnorr-family sigma protocol over a linear pairing relation (see alberta-buck-identity.org, "One-Time (PS Signature + NIZK Verification at Registration)") and follows the same completeness / soundness / ZK template as Theorems 3-5. A separate write-up at the same level of detail is the natural next step.
- *Circuit soundness* of the BUCK Notes spend circuits: Part IV proves that the in-circuit statements (CP-equality on E_{note} for A-spend; Schnorr PoK and signature verify for B-spend) imply the desired cryptographic guarantees *assuming* knowledge soundness of the underlying SNARK system. Soundness of the *proof system itself* (Groth16 / PLONK / STARK) depends on the chosen proof system's trusted setup (if any) and circuit correctness; this is orthogonal to the contents of this document.
- *Collision-resistance and hiding* of the Poseidon commitment hash H_{cm} and pseudorandomness of the Poseidon nullifier PRF. Poseidon is modelled as a random oracle in the SNARK circuit for the double-spend and commitment-binding arguments; its concrete security on BN254 has been independently studied and is assumed here. See the Poseidon-hash cryptanalysis literature for current best attacks and parameter choices.